

# Multi-domain Network Slice Partitioning

Zhouxiang Wu<sup>1</sup>, Genya Ishigaki<sup>2</sup>, Riti Gour<sup>3</sup>, Congzhou Li<sup>1</sup>, and Jason P. Jue<sup>1</sup>

1. Department of Computer Science, The University of Texas at Dallas, Richardson, Texas 75080, USA

2. Department of Computer Science, San Jose State University, San Jose, CA 95192, USA

3. Department of Computer Electronics and Graphic Technology,  
Central Connecticut State University, New Britain, Connecticut 06050, USA

**Abstract**—In the context of multi-domain network slices, multiple domains need to work together to provide a service. The problem of determining which part of the service fits within which domain is referred to as slice partitioning. The partitioning of multi-domain network slices poses a challenging problem, especially when considering the privacy concerns of each domain. To approach the optimal partition solution while maintaining load balance between domains, a framework has been proposed. This framework not only generates partition plans with various characteristics but also employs a Graph Neural Network solver, which significantly reduces the plan generation time. The proposed approach is promising in generating partition plans for multi-domain network slices and is expected to improve the overall performance of the network.

**Index Terms**—Network Slice, Resource Allocation, Machine Learning, Reinforcement Learning, Graph Neural Network, Integer Linear Programming

## I. INTRODUCTION

Network slicing [1] has been developed to address users' diverse requirements by leveraging virtual network functions (VNF) and software-defined networking (SDN). This approach creates isolated network environments based on existing network infrastructure. Traditional network slices have typically been confined to single or two adjacent domains, which often fail to achieve globally optimal resource management. Consequently, multi-domain end-to-end network slicing has emerged as a solution.

Multi-domain network slices [2] are commonly managed by a global coordinator who orchestrates resources across various domains. If a domain operator shares their domain's detailed information with the global coordinator, the problem becomes akin to the conventional VNF embedding problem [3] but on a larger scale. However, in cases where domain operators do not divulge such information, the global coordinator must employ inference techniques to map VNFs to the respective domains.

Consequently, the current mapping problem encompasses two stages. In the first stage, the global coordinator partitions the multi-domain network slice into distinct segments and assigns each segment to a domain, considering that multiple domains may be capable of supporting a given VNF. During the second stage, the domain operators develop embedding plans based on their assigned segments. The global coordinator then selects a set of embedding plans and stitches them together to deliver end-to-end services. In this paper, our primary focus lies in the first stage: efficiently partitioning network slices to minimize resource costs.

Partitioning network slices presents several challenges. The first challenge involves minimizing resource costs, which can be divided into two components: the resource cost within the domain and the resource cost between domains. Placing all elements within a single domain would undoubtedly reduce the resource cost between domains to zero; however, this would result in a high resource cost within the domain. Striking a balance between these two types of costs is essential. Another challenge concerns efficiency. As the global coordinator must perform the partitioning task for each incoming network slice request, a lengthy process time would contribute to increased queueing wait times for establishing and deploying new network slices. Additionally, maintaining load balance among domains is critical, as imbalanced arrangements can lead to resource waste.

Our study presents a formalization of the partition problem using an Integer Programming approach. Furthermore, we propose three distinct methodologies to address this problem. The remainder of this paper is structured as follows: Section II provides a summary of the related work. Section III presents the problem formulation. In Section IV, we propose three algorithms to address the problem. Simulation experiments and result analyses are conducted in Section V. Lastly, we draw our conclusions in Section VI.

## II. RELATED WORK

In previous work [4], we designed an architecture that enabled the global coordinator to select allocation plans generated by domain operators. However, that approach employed a fixed partitioning strategy. In this paper, we transition from a fixed strategy to a more dynamic and objective-oriented partitioning plan. There is a dearth of literature on the subject of network slice partitioning. Therefore, in this study, we provide an overview of the relevant research pertaining to the embedding of multi-domain Virtual Network Function Forwarding Graphs (VNF-FGs) and the associated load-balancing strategies.

In [5], the authors note the complexities and challenges associated with multi-domain non-cooperative scenarios. They propose a deep reinforcement learning-based approach for VNF-FG embedding to address these issues. The results reveal insights into the behavior of non-cooperative domains and demonstrate the efficiency of their approach, which enables automatic inter-domain load balancing. The paper [6] proposes a model of the adaptive and dynamic Virtual Network

Function - Forwarding Graph (VNF-FG) allocation problem, an integer linear programming (ILP) optimization problem, and a heuristic algorithm for allocating multiple VNF-FGs. They also propose a decentralized optimization approach for cooperative multi-operator scenarios. The results confirm that the proposed algorithms optimize network utilization while limiting the number of VNF reallocations that could interrupt network services. The paper [7] proposes a coordination-free algorithm that supports non-functional dependencies and offers strong eventual consistency. The algorithm has two variants, one preventive and one corrective, and the paper proves the correctness of both variants. The proposed algorithm outperforms the state of the art in terms of consistency and allowing orchestrators to reject ongoing reconfigurations without a significant impact on performance.

### III. PROBLEM FORMULATION

#### A. Multi-domain Network Infrastructure

The aim of this study is to investigate scenarios in which multiple domains collaborate to provision an end-to-end network slice. We assume that each type of domain is both contiguous and unique. To represent these domains, we use a list with  $M$  elements, where  $M$  is the number of domains. For example, there might be three distinct but contiguous domains: RAN, edge, and core. Our work can be extended easily to scenarios with multiple domains of each type by integrating parallel domains. Each domain is composed of non-disclosable resources, such as computation nodes and data transmission links. Importantly, the topology information and resource allocation within each domain are not accessible to the global coordinator due to access restrictions. Instead, each domain  $m \in M$  provides the global coordinator with estimates of the costs for CPU, RAM, bandwidth, and latency for its supported Virtual Network Functions (VNFs), denoted as  $c_{CPU}(m)$ ,  $c_{RAM}(m)$ ,  $c_{Link}(m)$ , and  $l(m)$ , respectively. Using this information, the global coordinator provisions the network slice partition plans across multiple domains. Domains also disclose their in-bound and out-bound points to the global coordinator. Furthermore, the global coordinator has access to information on the bandwidth on links between different domains and the associated costs per unit of bandwidth on an inter-domain link between domain  $m_i$  and  $m_j$ , which is given by  $c_{Link}(m_i, m_j)$ .

#### B. VNF-FG

We make the assumption that the network slice can be accurately represented as a VNF-FG, denoted by  $G$ . This VNF-FG maintains a set of nodes, denoted by  $N$ , and a set of edges, denoted by  $E$ . Each node  $n \in N$  is characterized by two numerical values:  $CPU_n$  and  $RAM_n$ , which respectively represent the required level of CPU and RAM for the corresponding VNF. The edge  $e \in E$  is characterized by the required bandwidth necessary for successful data transmission between two VNFs in the network slice.

#### C. VNF-FG Partitioning Problem Formulation

The partitioning problem can be formulated as a node classification problem, where a binary variable  $X_n^m$  indicates whether node  $n \in N$  is allocated to domain  $m$ . Specifically,  $X_n^m$  is defined as:

$$X_n^m = \begin{cases} 1, & \text{if } n \text{ is allocated to } m \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

The objective is to minimize the cost generated by the domains, as well as the cost generated by inter-domain connections. The cost generated by the domains can be formulated as a function  $DC$ :

$$DC = \sum_{n \in N} \sum_{m \in M} X_n^m \times \{c_{CPU}(m) \times CPU_n + c_{RAM}(m) \times RAM_n\}. \quad (2)$$

Similarly, the link cost generated by domains can be formulated as a function  $DL$ :

$$DL = \sum_{n_i \in N} \sum_{n_j \in N} \sum_{m \in M} X_{n_i}^m \times X_{n_j}^m \times bw(n_i, n_j) \times c_{Link}(m). \quad (3)$$

The inter-domain link cost can be formulated as a function  $IC$ :

$$IC = \sum_{n_i \in N} \sum_{n_j \in N} \sum_{m_i \in M} \sum_{m_j \in M \setminus \{m_i\}} X_{n_i}^{m_i} \times X_{n_j}^{m_j} \times bw(n_i, n_j) \times c_{Link}(m_i, m_j). \quad (4)$$

To formalize the objective, we define a function that minimizes the weighted sum of the three cost functions:

$$\min DC + DL + IC. \quad (5)$$

#### D. Load balance

The objective function previously presented does not take into account load balancing among the different domains. To address this issue, we propose a new metric that quantifies load balancing for the resources in the system. Given that the bandwidth within a domain is assumed to be sufficient, and RAM is relatively inexpensive compared to CPU, our focus is on the load balance of the CPU resource among different domains.

To achieve load balancing, we assume that CPU resources are provided by the number of cores, and that resources assigned to one network slice are dedicated and isolated. We use  $num(m)$  to denote the number of CPUs occupied by existing slice requests in domain  $m$ . First, we count the total number of CPUs required by deployed network slices and incoming slices, denoted by  $total$ , by traversing the deployment history.

Next, we calculate the ratio,  $r_m$ , between the number of Virtual Network Functions (VNFs) hosted by domain  $m$  and the total number of VNFs for every domain. This produces a

distribution that we use to compute entropy  $H$ , which serves as the metric for load balancing. A higher value of  $H$  indicates a better load balance for the system.

The equations used for this metric are as follows:

$$\begin{aligned} total &= \sum_m num(m) + \sum_n CPU_n \\ r_m &= (num(m) + \sum_{n \in N} X_n^m \times CPU_n) / total \\ H &= -(\sum_{m \in M} r_m \times \log(r_m)) \end{aligned} \quad (6)$$

In certain scenarios, it may not be desirable to distribute Virtual Network Functions (VNFs) evenly across all domains. For instance, it may be preferable for the Cloud domain to shoulder a larger proportion of the load. In such cases, instead of comparing to a uniform distribution, a predefined distribution  $P$  can be established, and load balancing can be optimized based on the Kullback-Leibler (KL) divergence between the current distribution and the predefined distribution.

The predefined distribution  $P$  can be expressed as a set of probabilities  $p_m$  for each domain  $m \in M$ . The KL divergence between the current distribution and the predefined distribution can be formulated as follows:

$$\begin{aligned} P &= \{p_m, \forall m \in M\} \\ KL &= -(\sum_{m \in M} r_m \times \log(\frac{p_m}{r_m})). \end{aligned} \quad (7)$$

This approach allows for a more flexible and targeted load balancing strategy that can accommodate specific distribution preferences or requirements. The KL divergence, a measure of the difference between two probability distributions, is employed as an indicator of load balance. The objective of load balancing is to align the distribution of CPU demandings among the domains with a predefined distribution  $P$ , such that the KL divergence between the current distribution  $R$  and  $P$  is minimized. The closer  $R$  is to  $P$ , the lower the KL divergence will be.

To align the distribution with a discrete uniform distribution,  $p(m) = \frac{1}{|M|}$  can be set. The load balancing problem can be formulated as an optimization problem with the following objective function:

$$\min \alpha DC + \gamma DL + \beta IC + \delta KL, \quad (8)$$

where  $\delta$  is a weighting coefficient that reflects the relative importance of load balancing.

The four objective functions, namely  $DC$ ,  $DL$ ,  $IC$ , and  $CE$ , have different scales, which may result in an imbalance during optimization. To address this issue, we utilize the min-max algorithm to standardize  $DC$ ,  $DL$ , and  $IC$ .

$$\begin{aligned} DC_{min} &= |N| \times (c_{CPU}(m_i) + c_{RAM}(m_j)) \\ \text{where } m_i &= \operatorname{argmin}_{m \in M} c_{CPU}(m) \end{aligned} \quad (9)$$

$$\begin{aligned} m_j &= \operatorname{argmin}_{m \in M} c_{RAM}(m) \\ DC_{max} &= |N| \times (c_{CPU}(m_i) + c_{RAM}(m_j)) \\ \text{where } m_i &= \operatorname{argmax}_{m \in M} c_{CPU}(m) \end{aligned} \quad (10)$$

$$m_j = \operatorname{argmax}_{m \in M} c_{RAM}(m)$$

$$\widehat{DC} = \frac{DC - DC_{min}}{DC_{max} - DC_{min}} \quad (11)$$

We adopt a similar normalization strategy for  $DL$  and  $IC$ , resulting in  $\widehat{DL}$  and  $\widehat{IC}$ . Although the theoretical value of KL divergence can be infinitely large, empirical evidence suggests that it typically ranges around 1. If the value of KL divergence is too large, then it is crucial to minimize it in order to align the distribution with the predefined distribution. Hence, the final objective function that we utilize is as follows:

$$\min \alpha \widehat{DC} + \beta \widehat{DL} + \gamma \widehat{IC} + \delta KL, \quad (12)$$

where  $\alpha$ ,  $\gamma$ ,  $\beta$ , and  $\delta$  are hyperparameters used for generating partition plans with different characteristics.

### E. Constraints

First, the indicator variable  $X_n^m$  can only take values of 1 or 0. Second, each function can be assigned to only one domain. Third, if there is an edge between node  $i$  and node  $j$ , then the index of the domain to which node  $i$  is assigned should be less than or equal to the index of the domain to which node  $j$  is assigned. This is because we assume that the VNF-FG is a Directed Acyclic Graph (DAG). We formalize the optimization problem as follows:

$$\begin{aligned} &\min \alpha \widehat{DC} + \gamma \widehat{DL} + \beta \widehat{IC} + \delta KL \\ \text{s.t. } &X_n^m \in \{0, 1\} \quad \forall n \in N \text{ and } 1 \leq m \leq M \\ &\sum_{n \in N} \sum_{m \in M} X_n^m = 1 \\ &X_{n_i}^{m_i} \times X_{n_j}^{m_j} = 0 \quad \forall (n_i, n_j) \in E \text{ and } m_i > m_j \end{aligned} \quad (13)$$

## IV. METHODOLOGY

### A. Heuristic

1) *Approximate Integer Linear Programming*: To linearize the quadratic terms in the objective function and constraints, we employ a binary integer quadratic programming (BIQP) reformulation. The fundamental approach involves introducing additional binary variables and linear constraints to replace the quadratic terms present in the original constraints and the objective function.

Another challenge that arises is the linearization of the KL divergence term, specifically the  $r_m \log r_m$  component in Equation 7. In order to approximate  $r_m \log r_m$ , we utilize the second-order Taylor expansion of  $r_m \log r_m$  at  $r_m = a$ :

$$r_m \log r_m \approx a \times \log(a) + (r_m - a) \times (\log(a) + 1) + \frac{(r_m - a)^2}{2 \times a}. \quad (14)$$

For our project, we set  $a = 0.3$ .

2) *Integer Programming: branch and bound*: Since the approximation of KL divergence sacrifices accuracy, we also employ another heuristic algorithm: branch-and-bound, to solve the problem. The fundamental idea of branch-and-bound is to divide the problem into smaller subproblems and use bounds to eliminate those subproblems that cannot possibly contain the optimal solution. The branch-and-bound

algorithm can be enhanced using various techniques such as cutting planes, heuristics, and symmetry breaking. While branch-and-bound is a powerful technique for solving integer programming problems, it can be computationally expensive for large-scale problems or those with complex constraints.

### B. Graph Neural Network

1) *Model*: The linear approximation entails the optimization of  $O(|V|^2)$  variables, resulting in a high time complexity for the branch and bound algorithm without guaranteed performance. As the global coordinator periodically devises the partition plan, the VNF-FG exhibits a similar structure. Consequently, we introduce a machine learning algorithm utilizing a graph neural network (GNN) for partitioning the VNF-FG, which we refer to as GNNP.

A fundamental framework of GNN is message passing. Each node in the graph is initialized with its feature vector, constituting the first layer. Subsequently, each node gathers information from its neighboring nodes, generating a new vector. These newly generated vectors form the second layer. By following this procedure,  $k$  layers are produced, where  $k$  is a hyperparameter that determines the GNN model's complexity. Numerous aggregation methods are available; however, we opt for the Graph Convolutional Network (GCN) [8]. Generally, it is assumed that the vectors from the final layer encompass all topological information. Therefore, we transform the vectors from the last layer into the designated domain. The comprehensive algorithm is presented in Algorithm 1.

---

#### Algorithm 1 GNN Partitioner

---

##### Input:

VNF-FG  $G = (N, E)$ , List of Domains  $M$ , the number of layers  $K$

##### Output: Assignment for each node $n \in N$

- 1: Initialize the feature vector for each node  $i$   $x_i^{(0)}$
- 2: **for**  $k$  in  $1, \dots, K$  **do**
- 3:

$$x_i^{(k)} = \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\deg(i)} \cdot \sqrt{\deg(j)}} \cdot (W^\top \cdot x_j^{(k-1)} + b)$$

4: **end for**

5: Embedding vector for node  $i$ :

$$x_i = \text{SoftMax}(W \cdot x_i^K + b)$$

6:  $X_i^m =$  the  $m^{\text{th}}$  element of  $x_i$

7: **return**  $X_i^m$

---

2) *Optimization*: As the range of the  $x_i$  cannot be directly controlled, constraints are incorporated into the objective function by employing relatively large coefficients. The first constraint is relaxed from a binary variable to a continuous variable ranging from 0 to 1. By virtue of the SoftMax function's definition, the second constraint is inherently satisfied.

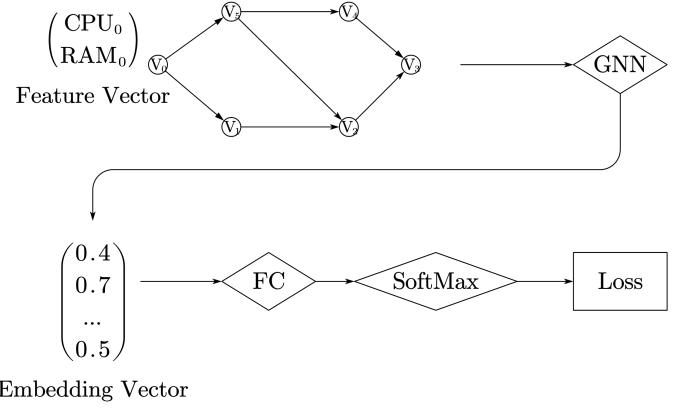


Fig. 1. GNN Partitioner

Consequently, the objective function can be formulated as follows:

$$\begin{aligned} \min \quad & \alpha \widehat{DC} + \gamma \widehat{DL} + \beta \widehat{IC} + \delta KL + \\ & \mu (X_{n_i}^{m_i} \times X_{n_j}^{m_j} = 0 \quad \forall (n_i, n_j) \in E \text{ and } m_i > m_j), \end{aligned} \quad (15)$$

where  $\mu$  is a sufficiently large value to ensure that the solution satisfies the third constraint. Given that this is not a conventional supervised learning task, backpropagation need only be performed based on the direct loss value. The complete procedure is depicted in Fig. 1.

## V. EXPERIMENTS

### A. Environment Setup

To represent 200 slices' VNF-FG, we generated 200 random DAGs. Each graph had the freedom to select its number of nodes from the set  $[10, 15, 20]$  and its number of edges from the set  $[15, 30, 60]$ . Each node could choose its CPU level from the set  $[2, 4, 8, 16]$  and its RAM level from the set  $[8, 16, 32, 64]$ . Each edge had the flexibility to select its bandwidth level from the set  $[100, 200, 500, 1000]$ . An example of the DAG is depicted in Figure 2. To train the GNNP model, we generated an additional 800 DAGs.

We created four domains: RAN domain, edge domain, core domain, and cloud domain, each with decreasing resource costs per unit. The CPU costs per unit for these domains were 100, 50, 20, and 5 respectively, while the RAM costs per unit were 10, 5, 2, and 1, respectively. The bandwidth cost per unit were 1, 0.5, 0.2, and 0.1 for RAN, edge, core, and cloud domains respectively. Additionally, we assigned an inter-domain cost per unit as 10, 5, and 2 for domain pairs (RAN, edge), (edge, core), and (core, cloud), respectively. The target CPU distribution for these domains was 0.1, 0.2, 0.3, and 0.4 for RAN, edge, core, and cloud domains respectively.

### B. Metrics

1) *Total Resource Cost*: Due to the significant variations in cost between different VNF-FGs, which may differ in the number of nodes and edges, we chose to compare algorithms

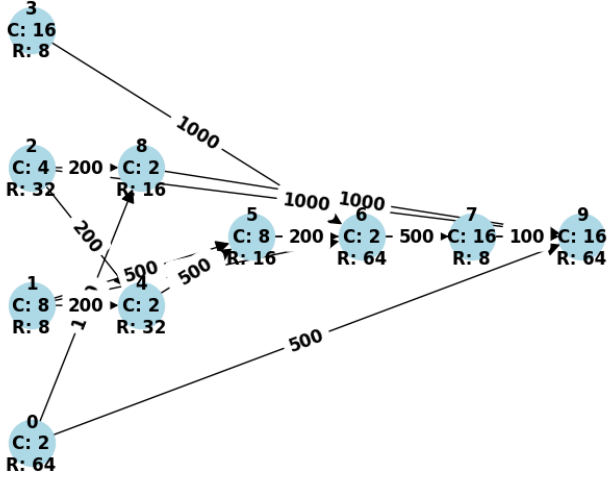


Fig. 2. Example of VNF-FG

based on the summation of costs for all 200 VNF-FGs, rather than their average cost.

2) *Divergency from Target Distribution*: The total resource cost calculation does not incorporate any deviation from the target distribution. Our objective is to evaluate the degree of alignment between the distribution generated by algorithms and the target distribution.

### C. Evaluation

1) *GNNP Model Selection*: We partitioned the supplementary 800 VNF-FGs into two distinct sets: 700 for training and 100 for validation purposes. The subsequent hyperparameter tuning procedure was conducted under a fixed set of parameters, specifically  $\alpha$ ,  $\beta$ ,  $\gamma$ , and  $\delta$ .

- **Latent Variable Size**: Increasing the size of the latent variable results in a higher number of parameters within the model. Given that our initial feature dimensionality is only 2, it is prudent to avoid using an excessively large latent variable size. In our study, we evaluated the performance of the model with latent variable sizes of 5, 10, 25, and 50. The number of layers is fixed to 3. The results are shown in Fig. 3<sup>1</sup>. Based on the experimental outcomes, a latent variable size of 10 appears to strike an optimal balance between model stability, performance, and computational complexity.
- **The number of layers**, denoted as  $k$ , plays a crucial role in determining the performance and complexity of the model. As  $k$  increases, each node attains a more comprehensive global perspective, which in turn augments the model's complexity. We experiment with four distinct values for  $k$ : 1, 3, 5, and 7. The outcomes of these experiments are illustrated in Figure 4.

<sup>1</sup> All of the boxplots in this paper are drawn based on 50 experiments. The circles denote the outlier. The upper bound of the vertical line, the upper bound of the box, the middle horizontal line within the box, the lower bound of the box, and the lower bound of the vertical line represent the minimum, the first quartile, the median, the third quartile and maximum, respectively.

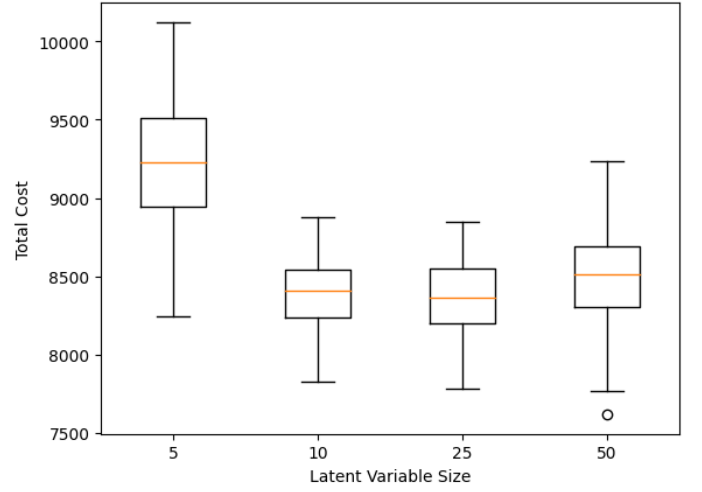


Fig. 3. Latent Variable Size Tuning

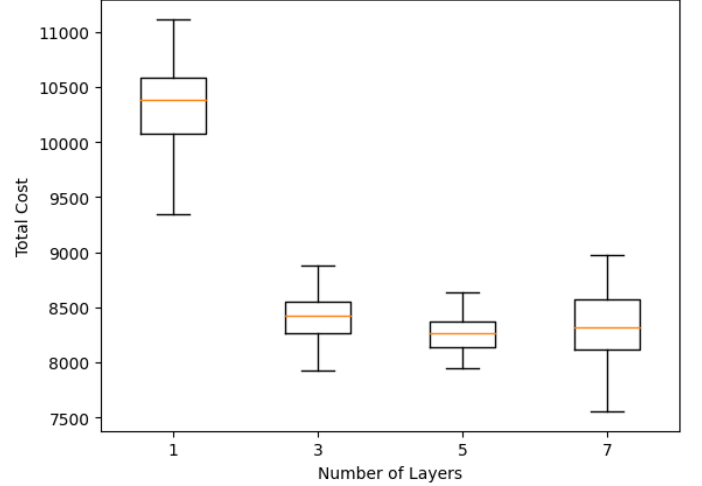


Fig. 4. Number of Layers Tuning

A prevalent observation indicates that, as the complexity of the model increases, there is a corresponding rise in average performance. Nonetheless, this increased complexity may also contribute to overfitting and an escalation in variance, resulting in diminished stability.

2) *Algorithm Comparasion*: We conduct a comparative analysis of three algorithms: Graph Neural Network for Partitioning (GNNP), Approximate Integer Linear Programming (AILP), and Branch and Bound (BnB). The results are presented in Table I. AILP demonstrates the most favorable average total cost, but it exhibits the least congruence with the target distribution. This discrepancy could be attributed to the approximation of the Kullback-Leibler (KL) divergence within the objective function. In contrast, BnB necessitates the longest inference time, and its performance is not guaranteed. GNNP produces outputs that align closely with the target distribution and boasts the shortest inference time. However, GNNP demands a comparatively extended training duration.

TABLE I  
ALGORITHM COMPARASION

|                    | GNNP        | AILP        | BnB  |
|--------------------|-------------|-------------|------|
| Total Cost         | 8539        | <b>8316</b> | 9539 |
| KL-divergence      | <b>0.05</b> | 0.18        | 0.08 |
| Inference Time (s) | <b>0.2</b>  | 2.4         | 5.3  |

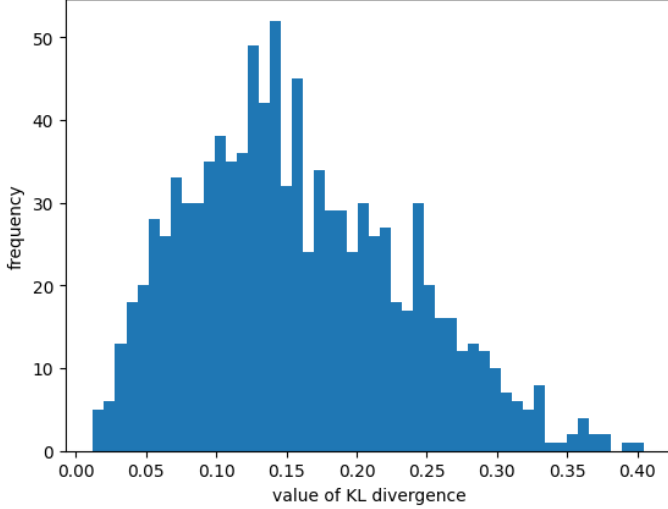


Fig. 5. KL divergency value distribution

#### D. Ablation Study

1) *Effects of Standardization Cost Items in Objective Function:* Initially, we must verify the scale of the Kullback-Leibler (KL) divergence. We sample  $10^4$  random distributions and calculate the KL divergence between them and the target distribution. The distribution of KL divergence values is depicted in Figure 5. In order to align the KL divergence item with other uniformed components in the objective function, we scale up the KL divergence item by a factor of two. Utilizing the GNNP algorithm, the KL divergence value is 0.249 before the adjustment and 0.186 after the adjustment.

If we do not standardize other terms in the objective function, the impact of the KL divergence item becomes negligible. This is due to the larger scale of other terms compared to the value of KL divergence. Without standardizing the other items, the average cost of partition plans generated by the GNNP algorithm is 8492, and the KL divergence is 0.58.

2) *Effect of  $\alpha$ ,  $\beta$ ,  $\gamma$ ,  $\delta$ :* By constraining the value of each term between 0 and 1, we can adjust the coefficients of individual items to generate different partition plans with varying tendencies. We provide two examples, both generated using the GNNP algorithm.

In the first example, we double the coefficient for the CPU cost. Before doubling, the average CPU cost is 3217, which drops to 2793 after doubling. Simultaneously, the ratio of CPU in the cloud domain increases from 0.51 to 0.56 after the adjustment.

In the second example, we double the coefficient for the inter-domain bandwidth cost. The resulting distribution be-

comes  $[0.17, 0.31, 0.35, 0.17]$ . As observed, most nodes are situated in the first three domains to lower the inter-domain cost, and the KL divergence is 0.134.

#### VI. CONCLUSION

In this paper, we explore the problem of multi-domain end-to-end network slice partitioning. We define resource costs and incorporate load balancing among different domains into the objective function. We propose two heuristic algorithms and one machine learning-based algorithm, comparing them using various metrics. Additionally, we tune the weights for different terms and demonstrate their effects. By adjusting the assigned weights, we can generate partition plans with distinct characteristics. The latency constraints were not considered in our algorithm. To incorporate them, the domains' ability to offer latency for each type of VNF can be assumed, and a constraint can be imposed on the total processing delay. This constraint can be treated as another cost element in the objective function. Our algorithm can also handle hard assignments where certain VNFs can only be assigned to specific domains. In such cases, the indicator variable for the VNF can be set to 1 for the assigned domain and 0 for all other domains without compromising the original framework. In future work, we aim to integrate the intelligent partitioning algorithm with our proposed multi-domain network slice provisioning architecture, to achieve more efficient resource utilization.

#### ACKNOWLEDGMENT

This work was supported in part by the National Science Foundation under Grant No. CNS-2008856.

#### REFERENCES

- [1] K. Samdanis, X. Costa-Perez, and V. Sciancalepore, "From network sharing to multi-tenancy: The 5g network slice broker," *IEEE Communications Magazine*, vol. 54, no. 7, pp. 32–39, 2016.
- [2] T. Taleb, I. Afolabi, K. Samdanis, and F. Z. Yousaf, "On multi-domain network slicing orchestration architecture and federated resource control," *IEEE Network*, vol. 33, no. 5, pp. 242–252, 2019.
- [3] I. Afolabi, T. Taleb, K. Samdanis, A. Ksentini, and H. Flinck, "Network slicing and softwarization: A survey on principles, enabling technologies, and solutions," *IEEE Communications Surveys & Tutorials*, vol. 20, no. 3, pp. 2429–2453, 2018.
- [4] Z. Wu and J. P. Jue, "Reinforcement learning-based multi-domain network slice provisioning," *IEEE International Conference on Communications*, 2023.
- [5] P. T. A. Quang, A. Bradai, K. D. Singh, and Y. Hadjadj-Aoul, "Multi-domain non-cooperative vnf-fg embedding: A deep reinforcement learning approach," in *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pp. 886–891, 2019.
- [6] P. T. A. Quang, A. Bradai, K. D. Singh, G. Picard, and R. Riggio, "Single and multi-domain adaptive allocation algorithms for vnf forwarding graph embedding," *IEEE Transactions on Network and Service Management*, vol. 16, no. 1, pp. 98–112, 2019.
- [7] J. Castañeda Cisneros, S. E. Pomares Hernández, J. C. Pérez Sansalvador, L. M. Rodríguez Henríquez, S. Yangui, and K. Drira, "Coordination-free multi-domain nfv orchestration for consistent vnf forwarding graph re-configuration," *IEEE Transactions on Network and Service Management*, vol. 19, no. 4, pp. 5133–5151, 2022.
- [8] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.